

Evidence Report

Sealed · tamper-evident · independently verifiable.
Screen what your agents ingest. Seal what they found.

VERDICT · LEAD FINDING (CVSS SEVERITY)

MEDIUM · CVSS 6.0

MEDIUM RISK — max severity 6/10 across 3 classified source(s); lead signal: Revenue guidance raised to \$480M.

CRYPTOGRAPHIC SEAL

Ed25519 sig keyId 76af8b6912a90684a665fc8d3830a759
GEqQ50cZgeZWnAPQ...9m6jktDNCA=
RFC 3161 TSA digicert · 2026-05-30T21:50:07.000Z
serial 00b64bfeec0398642e4ea3926f94421202
SHA-256 hash 7a6574387601083a530b...bd67aabfc2bbe4c0
HMAC-SHA256 0b43f4f89aa5f749de61...53114d63d24ac63c
(internal integrity checksum)
method · HMAC-SHA256 + Ed25519 + RFC 3161 TSA



scan to verify

TARGET
https://...

LENS
all

FETCHED
2026-05-30T21:49:56.473Z

SEALED
2026-05-30T21:50:08.051Z

SOURCES
3

BLOCKED PRE-LLM
0

Per-source data bill of materials

One row per source this run ingested through Bright Data, with its sealed content SHA-256, the surface that routes the URL, the fetched-at stamp, deduplication status, and the per-layer L1/L2/L3 screening verdict. This is the provenance ledger an audit asks for: every byte that reached classification is named and hashed.

SOURCE URL	SURFACE	FETCHED	SHA-256	DEDUP	SCREEN
https://cheatsheetseries.owasp...revention.html	Web Crawl	21:49:56Z	36ebe725... 261797	UNIQUE	L1:REVIEW L2:REVIEW L3:ALLOW
https://meridian-logistics.example/investors/q3	Web Crawl	21:49:56Z	841296d1... 11ad06	UNIQUE	L1:· L2:· L3:·
https://nimbus.example/changelog/v4.2	Web Crawl	21:49:56Z	b86b580d... 9d1dd4	UNIQUE	L1:· L2:· L3:·

How to read this ledger

Content SHA-256	truncated head...tail; the FULL hash for every source lives in the evidence.json sidecar.
BD surface	derived from the URL shape (this run used cached fixtures, so no live surface is sealed); a request-shape mapping, not a sealed fact.
Dedup	from matching content hashes: two sources with the same hash are duplicates. A source with no sealed hash (DUP) was dropped by FORGE before classification; the byte total is on the CFO page.
L1 / L2 / L3	read verbatim from the sealed payload.decisions[]: L1 = DJL/prefilter regex (REVIEW-only on ingest) · L2 = injection-guard · L3 = alignment-check. "·" = that layer did not run for this URL.

This run: 3 unique · 0 duplicate (0.0% deduplicated, 0 bytes saved).

Three layers screen what your agents ingest

Injection defense — 3 layers, from the sealed decision log

Reconstructed from the sealed decisions[]. L1 regex is REVIEW-only (it seals a signal, never drops a doc); L2 is an opt-in detector; L3 AlignmentCheck holds the only BLOCK authority.

LIVE	L1 regex (DJL/prefilter) 2 REVIEW — REVIEW-only on ingest
LIVE	L2 injection-guard 1 REVIEW (Qwen/Qwen3Guard-Gen-8B)
LIVE	L3 alignment-check ALLOW (conf 0.95) — The content is an OWASP cheat sheet describing prompt injection preven...

L3 verdict — describing vs executing (the false-positive killer)

L3 answers one question — does the scraped text DESCRIBE an attack (documentation) or INSTRUCT the reading agent to EXECUTE one? Executing is dropped before classify; describing is kept:

✓ DESCRIBING → ALLOW (benign control, kept)

<https://cheatsheetseries.owasp.org/cheatsheet/Prompt-Injection-Prevention.html> · LIVE · deepseek/deepseek-v4-pro · conf 0.95

The content is an OWASP cheat sheet describing prompt injection prevention; it includes an example payload for educational purposes, not to instruct the agent t...

Signal → framework mapping (mapping aid, not endorsement)

An indirect injection caught in scraped HTML maps across the LLM/agentic threat catalogs. No standards body reviewed these mappings — they are a good-faith aid for your control matrix.

Coverage: full = pack-validated AND policy-covered · partial = one signal · none = none.

SIGNAL	FRAMEWORK / CONTROL	COVERAGE
Indirect prompt injection in scraped HTML (executing) — L3 BLOCK	OWASP LLM01:2025 Prompt Injection (Indirect)	✓ full
Agent instructed to exfiltrate secrets via a tool call	OWASP ASI01 Agent Goal Hijack · ASI02 Tool Misuse	✓ full
LLM-targeted injection technique (adversarial input)	MITRE ATLAS AML.T0051 LLM Prompt Injection	✓ full

Hand this to your SOC — STIX 2.1 export

The sealed decision log exports as STIX 2.1 bundles (indicator SDOs) for your SIEM/TIP:

```
synthex stix-export <evidence.json> > incident.stix.json
```

The screening already paid for itself

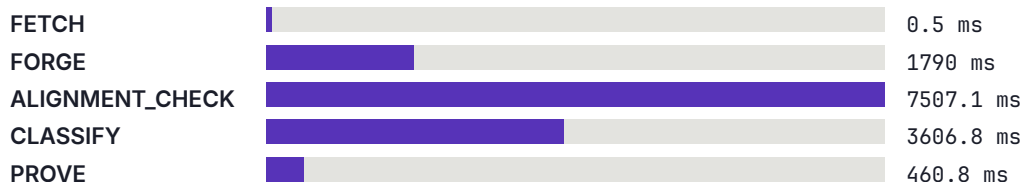
Token cost saved (deduplication)

FORGE deduplicates fetched documents by full SHA-256 before classification: every duplicate block removed is one fewer LLM call paid for. The ratio below is exact, not estimated.

Unique blocks classified 3
Duplicate blocks skipped 0
Input tokens saved ~0.0% (not paid to the model)
Bytes not re-processed 0 B

Latency per stage (wall-clock)

Measured by the pipeline (evidence.timings), outside the sealed payload so it never affects verification.



Total wall-clock: 13365.2 ms across 3 source(s)

The decision — cost of evidence vs cost of an incident

SYNTHEX · PER SIGNED EVIDENCE

\$0.0015

list price · \$0.75 / 500 URLs
marginal cost to seal what an agent ingested

AI-DATA INCIDENT · IBM 2025

USD 4.46M

avg AI-targeted breach · shadow-AI +USD 670K
Cost of a Data Breach Report 2025 (Ponemon)

This is a mapping, not an insurance figure: the USD 4.46M is IBM's measured industry average (ibm.com/reports/data-breach), not a price Synthex quotes or a loss it transfers. Synthex produces the signed, timestamped evidence that makes an incident defensible — it does not underwrite the risk. No third party reviewed this comparison.

The sealed log, mapped to the frameworks

This report is itself an evidentiary record: an automatic, timestamped log of the data lifecycle, sealed so any tampering is detectable. The matrix below maps that sealed log to the relevant articles of the EU AI Act — Regulation (EU) 2024/1689 — and to the NIST AI RMF functions. Each mapping is a good-faith aid, not an endorsement or certification; no framework body has reviewed this report.

EU AI Act — Regulation (EU) 2024/1689

✓ OK	Art 11 — Technical Documentation	Defenses, model ids and pipeline stages are documented on the Model & Pipeline Attestation page (Reg. (EU) 2024/1689, Annex IV scope).
✓ OK	Art 12 — Record-Keeping (logging)	Art 12 requires automatic logging over the system lifetime — this sealed, timestamped audit trail IS that log: inputs, pre-filter blocks and classifier outputs, sealed.
✓ OK	Art 13 — Transparency to Deployers	The verdict, sources and seal stack are disclosed to the deployer; system prompts are NOT exposed (Art 13 does not require it).

NIST AI RMF — GOVERN / MAP / MEASURE / MANAGE

▲ PARTIAL	GOVERN	Sealing + record-keeping policy is encoded in the pipeline; organisational governance is the deployer's responsibility (one signal of two).
✓ OK	MAP	Context mapped: 3 source URL(s) and their surfaces are recorded in the sealed payload.
✓ OK	MEASURE — MS-2.5	MS-2.5 (robustness): the 3-tier injection defense measures and records the catch on every scraped document; see the Security Briefing page.
▲ PARTIAL	MANAGE	Risks are surfaced and bounded (REVIEW/BLOCK) but residual-risk acceptance is a human decision outside this record (one signal of two).

What anchors this record

✓ OK	Tamper-evidence	Ed25519 signature (keyId 76af8b6912a90684a665fc8d3830a759) over the canonical payload; SHA-256 content hash 7a6574387601083a...c2bbe4c0. Any byte change breaks both.
✓ OK	Independent timekeeping	RFC 3161 trusted timestamp from digicert at 2026-05-30T21:50:07.000Z (serial 00b64bfeec03...). Proves WHEN — a third-party clock, not ours.
▲ PARTIAL	Transparency anchor	Not anchored in a public transparency log on this run (optional). The seal and timestamp above stand on their own.

This is an evidentiary record and a framework-mapping aid — NOT legal advice and NOT a certification. The seal proves WHEN these bytes existed and that they are unchanged, not that the underlying claims are true. Identity is self-signed, not a public trust-list certificate. Article applicability depends on whether the deploying system is classified high-risk under Regulation (EU) 2024/1689. No framework body has reviewed or endorsed this report.

Every model named, versioned, and hashed

The model-risk control question for any AI pipeline: which models actually ran, and under what build identity? Every model below is read from the sealed decision ledger — its id, version, and provider are part of the signed record, not asserted here. Each is labeled by its real run mode.

Models that ran (build identity, from sealed decisions)

STAGE	MODEL ID	VERSION	PROVIDER	MODE
L2 · injection guard	Qwen/Qwen3Guard-Gen-8B	qwen3guard-gen-v1	featherless	LIVE
Classify · tier model	function classifier (demo)	—	in-process	DEMO STUB
L3 · alignment reasoner	deepseek/deepseek-v4-pro	alignment-check-v1-7dcf5e0ac1d9	—	LIVE

Mode: LIVE = real model call this run · DEGRADED = model unreachable, failed safe to REVIEW · DEMO STUB = deterministic offline stand-in (this reproducible demo run). The full, untruncated model ids and hashes live in the sidecar evidence.json.

Build hashes & guard configuration

L2 guard model_hash	not pinned (stub — no live weights this run)
L2 guard_mode	featherless
L2 guard_score (this doc)	0.6 (REVIEW-capped — measured 40% benign FP, BLOCK disqualified)

Policy & rubric versions (content-addressed)

Each guard/classifier policy is content-addressed: the version string embeds a SHA-256 of the rubric text, so any change to the describing-vs-executing policy or the pre-filter rules yields a new, auditable id in the signed record.

DJL policy bundle	djl-v1-7c28e3783f10
Prefilter policy bundle	prefilter-v3-9c2c9b3d8c7e
L2 guard policy bundle	guard-v1-dd7c7e8b1d4c
L3 align-check rubric	alignment-check-v1-7dcf5e0ac1d9

Model-risk mapping (aid, not certification)

✓ OK	OCC/Fed SR 11-7 — model inventory & identity	every model id + version + policy hash is in the signed record
▲ PARTIAL	SR 11-7 — ongoing monitoring (degraded-state log)	fail-safe REVIEW + degraded:true sealed; not a full validation programme
✓ OK	NYDFS Part 500 (23 NYCRR 500) — audit trail	tamper-evident, timestamped decision ledger per run

This attestation maps the sealed model-identity record to SR 11-7 (Model Risk Management) and NYDFS Part 500 as a good-faith MODEL-RISK AID — it is a mapping, not an endorsement or certification. No regulator or framework body has reviewed these models or this report. Applicability depends on the deploying institution's own model-risk programme.

Risk evidence, mapped not endorsed

Synthex Risk Score — composite (0–100, severity + block-volume blend)

42 Composite band · MEDIUM
/ 100

How this number is computed (deterministic, reproducible)

```
maxSev    = max severity across all findings ..... 6/10 (CVSS base-score scale)
blockTerm  = min(blocked, 5) / 5 * 10 ..... 0.0/10 (blocked = 0)
score      = round( (maxSev*0.70 + blockTerm*0.30) * 10 ) = 42 / 100 → MEDIUM
```

Lead finding — CVSS severity (separate axis from the composite above)

▲ CVSS 6.0 / 10 · MEDIUM

CVSS axis is the worst single finding; the 0–100 composite above is a separate blend.

MEDIUM RISK — max severity 6/10 across 3 classified source(s); lead signal: Revenue guidance raised to \$480M.

Compliance framing — mapping aid (not endorsement)

▲ PARTIAL	NIST AI RMF — MEASURE	MS-2.5 (validity/robustness); GOVERN/MAP/MEASURE/MANAGE framing
▲ PARTIAL	EU AI Act 2024/1689	Art 15 robustness/cybersecurity · Art 9 risk management

DISCLAIMER

This Risk Score is computed by Synthex from the data in this report using the formula above — mapping, NOT endorsement; not an insurance rating; no third party reviewed this number. Severity follows the CVSS 0–10 scale; framing maps to NIST AI RMF and EU AI Act risk categories. See docs/compliance-mapping.md.

Verifiable evidence — to defend the highest-severity finding

Lead finding URL	https://meridian-logistics.example/investors/q3
Lead finding SHA-256	841296d14334f5eef55e...9ddfd71811ad06
Ed25519 keyId	76af8b6912a90684a665fc8d3830a759
Ed25519 signature	GEqQ50cZgeZWnAPQCp...9m6jktDNCA==
RFC 3161 TSA	digicert · 2026-05-30T21:50:07.000Z · serial 00b64bfe...421202
Report SHA-256	7a6574387601083a530b...67aabfc2bbe4c0
Full values in	SYNTHEX-EVR-7A657438.evidence.json

3 questions this evidence raises

1. Which of the 3 classified source(s) most changes our all exposure, and who owns the response?
2. Do the surfaced signals cross our escalation threshold under the all policy?
3. Is the sealed evidence (hash + timestamp + signature) sufficient to defend the highest-severity finding in an audit?

What this evidence does not prove

Stating the limits is the point. This page is the boundary of the claim: read it before you rely on the seal. Everything outside these lines is out of scope by design.

What the seal does not prove

- ✗ **It does not prove the scraped claims are true.**
The RFC 3161 timestamp proves WHEN these bytes existed and that they are UNCHANGED since — not that any price, headline, or finding inside them is accurate. Truth of the content is the source's, not the seal's.
- ✗ **It does not prove the source was authentic or uncompromised.**
If the scraped page itself was spoofed, poisoned, or served different bytes to the crawler, the seal faithfully records those bytes. Integrity of capture is not authenticity of origin.

What Synthex does not cover

- ✗ **Not a public-trust certificate.**
Identity is SELF-SIGNED with an Ed25519 key (the keyId is publishable) — it is NOT a certificate from a public trust list (no WebPKI / CA-anchored chain). A verifier trusts the publisher's pinned key, not a browser root store.
- ✗ **L1 screening is heuristic regex, not a formal proof.**
Layer-1 (78 DJL + 32 prefilter rules) is pattern-matching, REVIEW-only on ingest — it flags, it does not formally verify safety, and it never silently drops a benign document (measured below).
- ✗ **Framework mappings are a mapping aid, not certification.**
OWASP / MITRE ATLAS / NIST AI RMF / EU AI Act references are good-faith mappings, not endorsements. No standards body reviewed these numbers or certified this report.

Measured guard false positives (n=5 benign security corpus)

Every page in the corpus is benign by construction (security writing that DESCRIBES attacks), so any REVIEW or BLOCK is a false positive. Numbers are measured, not asserted — source: docs/guard-fp-measurement.md. n=5 is indicative, a floor, not a certified rate.

L1 pipeline (regex, REVIEW-only)	0 / 5 dropped	After the D5 fix, no benign doc is dropped; BLOCK-grade hits demote to REVIEW and stay sealed.
L2 Qwen3Guard-Gen-8B	≥ 2 / 5 FP	Benign FP > 20% in every run → DISQUALIFIED from BLOCK authority; its verdicts are REVIEW-capped.
L3 AlignmentCheck (deepseek)	0 / 5 false-BLOCK	Reads describing-vs-executing correctly; the only layer holding BLOCK authority on ingest.

Reading: L1 flags broadly but drops nothing; L2 is too trigger-happy on security content to hold BLOCK; L3 is the false-positive killer and the sole BLOCK authority. The conservative default is the measured one — no guard number here was hardcoded.

Reproduce every seal offline

Nothing here asks for trust. Each layer is checked with open tooling an auditor already has — no Synthex service, no network. The seal proves WHEN these bytes existed and that they are UNCHANGED, not that the claims are true. The downloadable sidecar evidence.json carries the FULL, untruncated hashes and signatures, so every head...tail value in this PDF has a path to the bytes you verify.

One command — verifies every present layer

```
$ synthex verify evidence.json
#   exit 0 → every present layer verified · non-zero → one failed
```

1 · RFC 3161 timestamp — proves WHEN (offline)

```
$ jq -r .seal.rfc3161Tsa.token evidence.json | base64 -d > synthex.tsr
$ openssl ts -verify -in synthex.tsr -sha256 \
    -digest 7a6574387601083a530bcd5e71618daa397c2dec93a7bdccbd67aabfc2bbe4c0 \
    -CAfile digicert-tsa-chain.pem
#   "Verification: OK"   exit 0   authority digicert · serial 00b64bfeec...94421202
```

2 · C2PA Content Credentials — provenance card

```
# Not present in this run (present-gated). When a manifest is attached:
$ c2patool evidence.c2pa.json --info
#   expect claimSignature.validated = true · dataHash.match = true   exit 0
```

3 · Sigstore Rekor v2 — offline inclusion + checkpoint

```
# Not present in this run (present-gated). When a bundle is attached, the
# inclusion + checkpoint are checked OFFLINE against the pinned log key:
$ rekor-cli verify --rekor_bundle evidence.rekor.json --pubkey rekor-log.pub
```

Ed25519 signature — verify against the published key

```
# keyId 76af8b6912a90684a665fc8d3830a759 (publishable public key in the sidecar)
$ jq -r .seal.signature.publicKey evidence.json | base64 -d > synthex.pub.der
$ jq -r .seal.signature.value evidence.json | base64 -d > synthex.sig
$ openssl pkeyutl -verify -pubin -inkey synthex.pub.der \
    -rawin -in payload.canonical.json -sigfile synthex.sig
#   "Signature Verified Successfully"   exit 0   identity is self-signed
```

The SHA-256 these checks must reproduce

[7a6574387601083a530bcd5e71618daa397c2dec93a7bdccbd67aabfc2bbe4c0](#)

Layers present in this run: SHA-256 · Ed25519 · RFC 3161 TSA · HMAC-SHA256. HMAC-SHA256 is an internal integrity checksum, not the headline.